

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	40% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	VIVETHA B	Reg. No.:	192424170
Section / Batch:	B.TECH AI&DS	Date:	

Code of Conduct

I VIVETHA B 192424170(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: VIVETHA B

Instructions

- Read each industry scenario carefully before answering.
- Analyze the given problem from a semantic analysis perspective.
- Provide structured and logical answers with proper justification.
- Support your analysis using examples from the given scenario wherever applicable.
- Use suitable diagrams, semantic representations, logical predicates, workflows, architectures, or tables whenever necessary.
- Clearly state assumptions if additional information is required.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze semantic interpretation of user queries, identify ambiguity in meaning representation, evaluate semantic processing limitations, and improve understanding in banking chatbot applications.	Q1
Syntax-Driven Semantic Analysis	Analyze multiple interpretations of spoken commands, evaluate parsing-related ambiguity, and assess semantic extraction in real-time voice assistant systems.	Q2
First-Order Predicate Calculus, Representing Linguistically Relevant Concepts, Syntax-Driven Semantic Analysis	Design a semantic processing architecture for healthcare reports, model relationships among entities and actions, represent linguistic concepts, and improve semantic extraction accuracy.	Q3

Annexure A – Industry Problem Solving Task

1. A company is developing an AI-powered customer support chatbot for banking services. The chatbot uses Context-Free Grammar (CFG) for parsing user queries. However, it faces issues such as:

Incorrect interpretation of ambiguous sentences

Failure to handle subject–verb agreement

Inefficient parsing for long queries

Example user query:

“Show me the transactions with the card from last month”

a) Analyze the ambiguity present in the given sentence using CFG

b) Evaluate the limitations of CFG in handling real-world conversational queries

c) Evaluate and propose an improved solution using PCFG, feature structures, or Earley parsing

d) Justify how your proposed method improves accuracy and efficiency in an industry setting

a) Analyze the Ambiguity Using CFG

The query is:

"Show me the transactions with the card from last month."

A basic CFG can represent the sentence as:

$S \rightarrow VP$

$VP \rightarrow V NP$

$NP \rightarrow Det N PP$

$PP \rightarrow P NP$

The sentence can be interpreted in more than one way.

Interpretation 1:

The user wants to see **transactions made with the card**, and those transactions are **from last month**.

Meaning:

Show [the transactions [with the card] [from last month]].

Here:

- **transactions** → main object
- **with the card** → specifies how the transactions were made
- **from last month** → specifies the time period

Interpretation 2:

The phrase **"from last month"** could potentially modify **"card"** rather than the transactions.

Meaning:

Show [the transactions [with the card from last month]].

This could imply that the card itself is associated with last month.

Therefore, the sentence has **syntactic attachment ambiguity**, particularly involving the prepositional phrases **"with the card"** and **"from last month."**

A simple CFG may generate multiple parse trees for such structures without knowing which interpretation is intended.

b) Limitations of CFG in Real-World Conversational Queries

CFG is useful for representing the grammatical structure of sentences, but it has several limitations in banking chatbots.

1. Ambiguity

CFG can generate multiple valid structures but does not inherently determine which interpretation is most likely.

For example:

"transactions with the card from last month"

can have multiple possible attachments.

2. No probability

A standard CFG treats grammar rules as equally valid. It does not know which interpretation is more likely based on previous user behavior or training data.

3. Subject–verb agreement

Basic CFG rules may accept grammatically incorrect combinations unless separate agreement rules are explicitly created.

For example:

"She show transactions."

A basic grammar may require additional rules to reject this sentence.

4. Long queries

As sentence length increases, the number of possible parse structures can increase significantly, making parsing computationally expensive.

5. Conversational language

Real users may type:

"show last month's card transactions pls"

or:

"Can you show me what I spent on my card last month?"

A strict CFG may not handle such variations effectively without many additional grammar rules.

6. Context understanding

CFG mainly represents syntax. It does not naturally understand banking concepts such as:

transactions, cards, account numbers, dates, payments, transfers, and balances.

c) Improved Solution Using PCFG, Feature Structures, and Earley Parsing

A better banking chatbot can combine **PCFG + Feature Structures + Earley Parsing**.

PCFG

A **Probabilistic Context-Free Grammar** assigns probabilities to grammar rules.

For example:

NP → NP PP [0.7]

NP → N PP [0.3]

The system can compare different parse trees and select the one with the highest probability.

For the query:

"Show me the transactions with the card from last month."

the PCFG can assign a higher probability to the interpretation where **"from last month"** modifies **transactions**, because that structure may be more common in banking queries.

Feature Structures

Feature structures can represent grammatical features such as:

Number = Singular/Plural

Person = First/Second/Third

For example:

User → Person = Third, Number = Singular

shows → Person = Third, Number = Singular

This allows the chatbot to enforce subject–verb agreement.

Feature structures can also represent semantic features such as:

Object = Transaction

PaymentMethod = Card

Time = Last Month

Thus, the query can be converted into a structured representation such as:

QUERY(Transaction, PaymentMethod=Card, Time=LastMonth)

Earley Parsing

The **Earley parser** is suitable for conversational queries because it can efficiently handle general CFGs, including grammars containing left recursion.

It can parse different sentence structures and identify possible interpretations.

Therefore, the improved architecture can be:

User Query → Earley Parser → PCFG Ranking → Feature Checking → Semantic Representation → Chatbot Response

d) How the Proposed Method Improves Accuracy and Efficiency

The proposed combination provides several advantages.

1. Better ambiguity resolution

PCFG assigns probabilities to possible interpretations and selects the most likely parse.

For example:

"transactions with the card from last month"

can be interpreted according to learned banking-language patterns.

2. Better subject–verb agreement

Feature structures ensure that grammatical features such as **person** and **number** are compatible.

For example:

She shows → Correct

She show → Incorrect

3. Efficient parsing

Earley parsing can handle complex and long sentences while supporting general CFG structures.

This makes it suitable for conversational banking queries.

4. Better semantic understanding

Feature structures can connect syntactic information with banking concepts.

For example:

"Show me the transactions with the card from last month."

can be represented as:

QUERY

Object → Transactions

Payment Method → Card

Time Period → Last Month

The chatbot can then retrieve the correct transaction information.

5. Improved customer experience

The system is more likely to understand different ways customers express the same request.

For example:

"Show my card transactions from last month."

"Can you show what I spent on my card last month?"

"I want last month's card transactions."

All can be mapped to the same intent:

QUERY(Transaction, Card, LastMonth)

Conclusion

A basic **CFG alone is insufficient for a real-world banking chatbot** because it primarily handles syntactic structure and cannot reliably resolve ambiguity, agreement, context, or conversational variations.

A combination of **PCFG, Feature Structures, and Earley Parsing** provides a stronger solution. **PCFG** helps select the most probable interpretation, **feature structures** handle grammatical agreement and structured information, and **Earley parsing** provides flexible parsing for complex queries.

Therefore, the proposed approach improves **accuracy, scalability, efficiency, and reliability**, making it more appropriate for an industry-level banking customer support system.

2. A voice assistant system (like Alexa/Siri-type applications) processes spoken commands using parsing techniques. The system currently uses top-down parsing, but faces:

Backtracking issues

Poor handling of incomplete or ambiguous input

Difficulty in real-time response

Example command:

“Book a flight to Delhi with a window seat”

a) Analyze how the given sentence can lead to multiple parse structures

b) Analyze the limitations of top-down parsing in this real-time application

c) Analyze how Earley parsing can improve handling of ambiguity and partial inputs

d) Compare and analyze the performance of top-down vs Earley parsing for such industry use cases

a) Multiple Parse Structures

The command is:

“Book a flight to Delhi with a window seat.”

The sentence can have different syntactic interpretations because the phrase **“with a window seat”** can attach to different parts of the sentence.

Parse Structure 1:

The phrase **“with a window seat”** describes the **flight**.

Meaning:

Book [a flight [to Delhi] [with a window seat]].

Here, the user wants to book a flight to Delhi and specifically wants a window seat.

Parse Structure 2:

The phrase **“with a window seat”** could be interpreted as an additional property associated with the booking request.

Meaning:

Book [a flight to Delhi] [with a window seat].

In practical terms, both structures lead to a similar interpretation, but syntactically the attachment can be ambiguous.

The ambiguity mainly occurs because **“with” introduces a prepositional phrase (PP)** that can attach to different parts of the sentence.

b) Limitations of Top-Down Parsing

Top-down parsing begins with the start symbol and attempts to generate the input sentence by expanding grammar rules.

For a voice assistant, this approach has several limitations.

1. Backtracking

When the parser chooses an incorrect grammar rule, it may need to backtrack and try another rule. This increases processing time.

2. Ambiguous sentences

When multiple interpretations are possible, top-down parsing may explore several alternatives before finding the correct structure.

3. Incomplete input

Voice assistants frequently receive incomplete commands because of speech interruptions, noise, or users changing their sentences.

For example:

“Book a flight to...”

A traditional top-down parser may struggle to determine the intended structure before the complete input arrives.

4. Real-time requirements

Voice assistants need to respond quickly. Excessive backtracking can increase latency.

5. Computational overhead

For long or highly ambiguous sentences, exploring multiple possible derivations can become expensive.

6. Conversational speech

Users may speak informally:

“Book Delhi... window seat... tomorrow.”

A strict top-down grammar may have difficulty processing such fragmented input.

c) How Earley Parsing Improves Ambiguity and Partial Input Handling

The **Earley parser** is a dynamic-programming parsing algorithm designed to handle general context-free grammars.

It improves the voice assistant system in several ways.

1. Handles ambiguity

Earley parsing can maintain multiple possible parse states instead of immediately committing to a single interpretation.

For:

“Book a flight to Delhi with a window seat.”

the parser can maintain possible interpretations involving the attachment of **“with a window seat.”**

2. Handles incomplete input

Earley parsing can maintain partial parsing states while the sentence is still being received.

For example:

“Book a flight to Delhi...”

The system can recognize that a valid sentence structure is already developing and wait for additional information.

3. Reduces unnecessary backtracking

Instead of repeatedly starting over when a particular interpretation fails, Earley parsing stores intermediate states and reuses them.

4. Supports general CFGs

Earley parsing can handle a wide range of context-free grammars, including grammars that are difficult for some other parsing strategies.

5. Suitable for speech applications

The ability to maintain partial hypotheses makes Earley parsing useful when spoken commands arrive incrementally.

d) Top-Down vs Earley Parsing for Industry Use

Top-Down Parsing

Top-down parsing is relatively simple and can be efficient when the grammar is small, the input is predictable, and ambiguity is low.

However, its performance can degrade when the input contains many possible interpretations. Backtracking can also increase latency.

For example:

“Book a flight to Delhi with a window seat.”

The parser may need to explore multiple possibilities for the **“with”** phrase.

Earley Parsing

Earley parsing uses dynamic programming to store intermediate parsing states. This prevents repeated work and allows multiple possible interpretations to be maintained.

It is particularly useful when:

Input is ambiguous

Input is incomplete

Grammar is complex

Real-time processing is required

Conclusion

For a simple voice assistant with a small and predictable grammar, **top-down parsing can be sufficient**.

However, for an industry-scale voice assistant that must handle **ambiguous, incomplete, long, and conversational commands**, **Earley parsing is generally more suitable**.

For the command:

“Book a flight to Delhi with a window seat.”

Earley parsing can maintain possible interpretations, process partial input, reduce repeated computation, and provide a more reliable parse for downstream intent detection and response generation.

Therefore, Earley parsing provides better flexibility and robustness for real-time voice assistant applications.

3. A healthcare company is developing an AI system to analyze patient medical reports and automatically extract critical information such as diagnosis, prescribed treatments, and follow-up actions.

The system must handle:

Complex and lengthy medical sentences

Ambiguity in sentence structure

Accurate subject-verb agreement

Relationships between medical actions and entities (sub-categorization)

Real-time processing for hospital use

Example input:

"The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai."

The current system, based only on basic Context-Free Grammar (CFG), fails to correctly interpret relationships and produces inconsistent outputs.

a) Design a complete NLP architecture integrating:

CFG for syntactic structure

Probabilistic CFG (PCFG) for ambiguity resolution

Feature Structures for agreement handling

Efficient parsing techniques

b) Develop a step-by-step workflow showing how the system processes the given medical sentence from input to structured output (diagnosis, action, location).

c) Create a method to:

Incorporate sub-categorization frames for medical verbs

Handle real-time data processing

Improve accuracy and scalability in a hospital environment

a) Complete NLP Architecture

A healthcare NLP system should use multiple NLP techniques instead of relying only on basic CFG.

Input Medical Report

The system first receives the patient's medical report as text or speech converted into text.

↓

Text Preprocessing

The system performs:

Tokenization → Sentence segmentation → Normalization

Medical abbreviations and terminology are preserved during preprocessing.

↓

Lexical Analysis

Each word is identified and assigned basic linguistic information.

For example:

doctor → noun

reviewed → verb

patient → noun

medication → noun

Chennai → location

↓

CFG-Based Syntactic Parsing

CFG is used to identify the basic grammatical structure.

For the given sentence, the structure can be represented as:

S → NP VP

The subject is:

The doctor who reviewed the patient last week

The main verb is:

recommends

The complement contains:

starting medication and scheduling a follow-up visit in Chennai

↓

PCFG-Based Ambiguity Resolution

A PCFG assigns probabilities to possible parse structures.

If the sentence has multiple possible interpretations, the PCFG selects the most probable parse based on patterns learned from medical language data.

This is useful for long sentences containing relative clauses and multiple prepositional phrases.

↓

Feature Structure Processing

Feature structures maintain grammatical information such as:

Person

Number

Tense

For example:

doctor → **Number = Singular**

recommends → **Number = Singular**

Therefore, the system verifies:

doctor → **recommends**

rather than an incorrect plural agreement.

↓

Sub-Categorization Analysis

Medical verbs require particular types of complements.

For example:

recommend → **NP/gerund clause**

schedule → **appointment/event**

diagnose → **disease/condition**

The system uses these frames to identify the relationships between medical actions and entities.

↓

Semantic Role Analysis

The system identifies roles such as:

Doctor → **Agent**

Patient → **Patient/Experiencer**

Medication → **Treatment**

Follow-up visit → Medical Action

Chennai → Location

↓

Information Extraction

Important information is extracted into structured fields:

Diagnosis

Treatment

Follow-up Action

Location

↓

Hospital Database / Clinical Decision System

The structured information can then be stored or sent to downstream hospital systems.

b) Step-by-Step Workflow for the Given Sentence

Input:

"The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai."

Step 1: Tokenization

The sentence is divided into individual tokens:

The | doctor | who | reviewed | the | patient | last | week | recommends | starting | medication | and | scheduling | a | follow-up | visit | in | Chennai

Step 2: Identify the Main Sentence Structure

The main structure is:

Subject + Verb + Complement

The main subject is:

The doctor

The main verb is:

recommends

The complement is:

starting medication and scheduling a follow-up visit in Chennai

Step 3: Identify the Relative Clause

The phrase:

"who reviewed the patient last week"

is a relative clause describing **the doctor**.

Its structure is approximately:

who → **Subject**

reviewed → **Verb**

the patient → **Object**

last week → **Time**

Therefore:

Doctor → **reviewed** → **Patient**

Step 4: Check Agreement

The subject is:

doctor → **Singular**

The main verb is:

recommends → **Third-person singular**

Therefore:

doctor recommends

is grammatically correct.

The feature structure can be represented as:

Doctor: Number = Singular

Recommends: Number = Singular, Person = Third

Therefore, the agreement check succeeds.

Step 5: Apply Medical Sub-Categorization

The verb:

recommends

introduces recommended actions.

The system identifies:

recommends → starting medication

and:

recommends → scheduling a follow-up visit

Thus, there are two recommended medical actions.

Step 6: Identify Medical Entities

The important entities are:

Doctor → Healthcare professional

Patient → Patient

Medication → Treatment

Follow-up visit → Medical follow-up action

Chennai → Location

Step 7: Generate Structured Output

The system can produce:

Diagnosis: Not explicitly mentioned

Treatment: Starting medication

Follow-up Action: Schedule a follow-up visit

Location: Chennai

Agent: Doctor

Patient: Patient

Time: Last week

The important point is that the system should **not invent a diagnosis**, because the sentence does not explicitly state one.

c) Method for Sub-Categorization, Real-Time Processing, Accuracy, and Scalability

1. Incorporating Sub-Categorization Frames

Medical verbs can be stored with sub-categorization frames.

For example:

recommend → recommend + medical action

prescribe → prescribe + medication + patient

diagnose → **diagnose** + **disease/condition** + **patient**

schedule → **schedule** + **appointment/visit**

administer → **administer** + **medication** + **patient**

When the parser encounters:

"recommends starting medication"

the system recognizes that **starting medication** is the recommended medical action.

This prevents the system from incorrectly treating **medication** as the subject or as an unrelated entity.

2. Handling Real-Time Processing

For hospital environments, the system should use an efficient processing pipeline.

The architecture can be:

Medical Input → **Preprocessing** → **CFG** → **PCFG** → **Feature Checking** → **Sub-Categorization** → **Semantic Analysis** → **Information Extraction** → **Hospital Database**

To improve speed:

Use efficient parsing algorithms

Earley parsing can be used for complex CFGs and ambiguous structures.

Use caching

Frequently encountered medical terms and grammatical patterns can be cached.

Use incremental processing

The system can process incoming text sentence by sentence instead of waiting for an entire medical report.

Use parallel processing

Different reports can be processed simultaneously.

Use optimized medical dictionaries

Common medical terms can be indexed for faster lookup.

3. Improving Accuracy

Accuracy can be improved by combining multiple sources of information:

CFG → Provides syntactic structure.

PCFG → Resolves syntactic ambiguity using probabilities.

Feature Structures → Handles person, number, tense, and agreement.

Sub-Categorization Frames → Identifies relationships between verbs and medical entities.

Medical Ontologies → Identify medical concepts and terminology.

Semantic Role Labeling → Identifies roles such as Agent, Patient, Treatment, and Location.

This combination is more reliable than using CFG alone.

4. Improving Scalability in Hospitals

For a large hospital environment, the system should support many reports and users simultaneously.

Scalability can be achieved through:

Cloud or distributed processing

Multiple NLP processing instances can analyze reports simultaneously.

Modular architecture

Each component can be independently updated or scaled.

Medical terminology database

A centralized terminology database can be shared across departments.

Continuous model improvement

The system can learn from validated medical reports and corrections made by healthcare professionals.

Security and privacy

Patient information must be protected through authentication, access control, encryption, and appropriate healthcare data-protection practices.

Final Structured Output for the Given Sentence

Input:

"The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai."

Diagnosis: Not specified

Treatment: Starting medication

Follow-up Action: Scheduling a follow-up visit

Location: Chennai

Agent: Doctor

Patient: Patient

Time: Last week

Conclusion

A basic CFG is not sufficient for complex healthcare reports because it mainly provides syntactic structure. A combined architecture using **CFG + PCFG + Feature Structures + Earley Parsing + Sub-Categorization Frames + Semantic Role Analysis** can handle ambiguity, agreement, complex medical relationships, and real-time processing more effectively.

This approach provides **better accuracy, faster processing, improved scalability, and more reliable structured medical information extraction** for hospital applications.

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Q. No. / Criteria Level	Understanding of Industry Problem	Application of Engineering Concepts	Solution Design	Use of Tools	Analysis	Professionalism	Sub. Total	Total %
1								
2								
3								

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____